

Дизайн и анализ на алгоритми - консултация

Станислав Игов (Стъката)

2025/2026

Задачи за консултация преди контролно 2: Обир на къщи и Брой строго нарастващи пътища в DAG.

Преговор

Нека си припомним петте стъпки за решаване на ДП задача:

1. **Дефинирай подзадачите.** Какво точно пресмята $dp[i]$ (или $dp[v]$, $dp[i][j]$ и т.н.)?
2. **Намери рекурентната връзка.** Как $dp[i]$ зависи от по-малки подзадачи?
3. **Определи базовите случаи.** Кой подзадачи знаем директно?
4. **Определи реда на пресмятане.** В какъв ред попълваме таблицата?
5. **Определи крайния отговор.** Коя клетка (или комбинация) дава отговора?

При всяка задача ключовият въпрос е: *кое е последното решение, взето в оптималния отговор?* Отговорът на този въпрос определя рекурентната връзка.

Задача 1: Обир на къщи (House Robber)

Условие: Вие сте професионален крадец, планираш да обере къщи покрай една дълга улица. Всяка къща съдържа определена сума пари, скрита в сейфа ѝ. Съседните къщи са свързани с обща охранителна система — ако се оберат две съседни къщи в една и съща нощ, се задейства аларма. Даден е масив $A[1..n]$ от неотрицателни цели числа, представящи сумите във всяка къща. Намерете максималната сума, която може да бъде открадната без задействане на аларма.

Мотивация

Това е класическа задача за оптимизация с ограничение „не вземай две съседни“. Алчният подход (например да избираме винаги най-голямата налична къща) не работи: при $A = [2, 1, 1, 10, 1, 1, 2]$ алчно бихме започнали с $A[4] = 10$, после ще ни остане да изберем най-много от позиции $\{1, 2, 6, 7\}$ при ограничението — ще загубим всякаква стойност, ако не сме внимателни.

Ключово наблюдение: За последната къща i имаме точно два изчерпателни и взаимно изключващи се избора — или я обираме, или не. Това е сигнал за ДП с просто едномерно състояние.

Стъпка 1: Дефиниция на подзадачата

Нека:

$dp[i]$ = максималната сума пари, която може да бъде открадната, използвайки само първите i къщи

Стъпка 2: Рекурентна връзка

За къща i имаме точно два избора:

- **Не обираме къща i :** Оптимумът идва изцяло от първите $i - 1$ къщи. Стойност: $dp[i - 1]$.
- **Обираме къща i :** Тогава задължително не сме обирали къща $i - 1$ (за да не задействаме алармата). Всички предишни къщи $\{1, \dots, i - 2\}$ са разрешени. Стойност: $dp[i - 2] + A[i]$.

Вземаме по-добрия от двата варианта:

$$dp[i] = \max(dp[i - 1], dp[i - 2] + A[i])$$

Стъпка 3: Базови случаи

$$\begin{aligned} dp[0] &= 0 && \text{(няма къщи — нулева сума)} \\ dp[1] &= A[1] && \text{(една къща — обираме я, тъй като } A[1] \geq 0) \end{aligned}$$

Стъпка 4: Ред на пресмятане и псевдокод

Попълваме таблицата отляво надясно: $i = 2, 3, \dots, n$.

```
HouseRobber(A[1..n]):
@1  if n = 0: return 0
@2  if n = 1: return A[1]
@3  dp[0..n] <- malloc(n+1){0}
@4  dp[0] <- 0
@5  dp[1] <- A[1]
@6  for i from 2 to n:
@7      dp[i] <- max( dp[i-1],           // случай 1: не обираме къща i
@8                  dp[i-2] + A[i] )    // случай 2: обираме къща i
@9  endfor
@10 return dp[n]
```

Отговор: $dp[n]$.

Сложност: $O(n)$ по време, $O(n)$ по памет. (Лесно се сваля до $O(1)$ памет, тъй като всяка стъпка използва само $dp[i - 1]$ и $dp[i - 2]$.)

Защо рекурентната връзка е коректна?

Ключовото наблюдение е, че оптималното решение *или* включва къща i , *или* не. Тези два случая са **изчерпателни и взаимно изключващи се**, следователно оптимумът е максимумът от двата сценария.

В **случай 1** (не обираме i): задачата се свежда директно до първите $i - 1$ къщи.

В **случай 2** (обираме i): къща $i - 1$ е забранена поради ограничението за съседство, но всички предходни $(1, \dots, i - 2)$ са разрешени. Задачата се свежда до първите $i - 2$ къщи плюс печалбата $A[i]$.

И в двата случая подзадачите са *строго по-малки* ($i - 1 < i$ и $i - 2 < i$), което гарантира липсата на циклични зависимости.

Коректност (индукция по i)

Твърдение: $dp[i]$ е максималната сума, постижима при обирање на подмножество от първите i къщи, без да се обират две съседни.

База ($i = 0$): Няма къщи; единствената допустима печалба е 0. $dp[0] = 0$.

База ($i = 1$): Една къща; обираме я. $dp[1] = A[1]$.

Индукционна стъпка ($i \geq 2$): Предполагаме, че $dp[j]$ е коректно за всички $j < i$. Нека OPT е оптимално допустимо подмножество от $\{1, \dots, i\}$.

- **Случай 1:** $i \notin OPT$. Тогава OPT е допустимо подмножество на $\{1, \dots, i - 1\}$ и е оптимално сред такива (иначе можем да го заменим с по-добро и да получим противоречие с оптималността на OPT). По индукционна хипотеза $|OPT| = dp[i - 1]$. Формулата взема $\max(dp[i - 1], \dots) \geq dp[i - 1]$.

- **Случай 2:** $i \in OPT$. Тогава $i - 1 \notin OPT$ (нарушаване на условието за съседство). Следователно $OPT \setminus \{i\}$ е допустимо подмножество на $\{1, \dots, i - 2\}$ и е оптимално сред такива (иначе го заменяме с по-добро и добавяме i обратно — противоречие). По индукционна хипотеза $|OPT \setminus \{i\}| = dp[i - 2]$, тоест $|OPT| = dp[i - 2] + A[i]$.

Тъй като двата случая са изчерпателни и формулата взема максимума от тях, $dp[i] = |OPT|$.

Пример

Нека $A = [2, 7, 9, 3, 1]$, т.е. $n = 5$.

i	0	1	2	3	4	5
$dp[i]$	0	2	7	11	11	12

Пресмятания:

- $dp[2] = \max(dp[1], dp[0] + A[2]) = \max(2, 0 + 7) = 7$
- $dp[3] = \max(dp[2], dp[1] + A[3]) = \max(7, 2 + 9) = 11$
- $dp[4] = \max(dp[3], dp[2] + A[4]) = \max(11, 7 + 3) = 11$
- $dp[5] = \max(dp[4], dp[3] + A[5]) = \max(11, 11 + 1) = 12$

Отговор: $dp[5] = 12$ — обираме къщи 1, 3, 5 (суми $2 + 9 + 1 = 12$).

Задача 2: Брой строго нарастващи пътища в DAG

Условие: Даден е насочен ацикличен граф $G = (V, E)$ с $n = |V|$ върха и $m = |E|$ ребра. Във всеки връх v е написано цяло число $\text{val}[v]$. Път $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k$ в G (т.е. $(v_t, v_{t+1}) \in E$ за всяко t) се нарича *строго нарастващ*, ако

$$\text{val}[v_1] < \text{val}[v_2] < \dots < \text{val}[v_k].$$

Намерете **общия брой** строго нарастващи пътища в G .

Конвенция: Пътищата с дължина 0 (от един връх, т.е. $k = 1$) се броят — по подразбиране те са строго нарастващи.

Мотивация

Наивният подход — да изброим всички пътища — е експоненциален: даже в DAG броят на пътищата може да е $\Omega(2^n)$. Но ние не искаме самите пътища, а само техния **брой**, което подсказва, че можем да преброим чрез ДП, без да изпробваме всички пътища поотделно.

Ключово наблюдение: Всеки строго нарастващ път с дължина ≥ 1 , започващ от v , започва с някое *първо ребро* $(v, u) \in E$ с $\text{val}[u] > \text{val}[v]$. Останалата част от пътя ($u \rightarrow \dots$) сама по себе си е строго нарастващ път, започващ от u . Това подсказва да дефинираме $dp[v]$ върху върховете.

Стъпка 1: Дефиниция на подзадачата

Нека:

$$dp[v] = \text{брой на строго нарастващите пътища, започващи от връх } v$$

(включително тривиалния път, състоящ се само от v).

Стъпка 2: Рекурентна връзка

Строго нарастващите пътища, започващи от v , се разделят на две групи:

- **Тривиалният път** (v) — точно 1 такъв.
- **Пътища с дължина ≥ 1** : класифицирани по първото си ребро (v, u) . За да е допустимо първото ребро, трябва $(v, u) \in E$ и $\text{val}[u] > \text{val}[v]$. Останалата част на пътя ($u \rightarrow \dots$) е произволен строго нарастващ път, започващ от u — и такива има точно $dp[u]$ на брой.

Сумирайки по всички допустими първи ребра:

$$dp[v] = 1 + \sum_{\substack{(v,u) \in E \\ \text{val}[u] > \text{val}[v]}} dp[u]$$

Краен отговор:

$$\text{Total} = \sum_{v \in V} dp[v]$$

(всеки път се брой точно веднъж — по върха, от който започва).

Стъпка 3: Базов случай

За връх v без изходящи ребра (терминален връх), сумата в рекурентната формула е празна и:

$$dp[v] = 1$$

(само тривиалният път). Това се хваща автоматично от формулата, така че не се налага отделна обработка.

Стъпка 4: Ред на пресмятане и псевдокод

$dp[v]$ зависи от $dp[u]$ за съседите u , до които има изходящо ребро. В топологическа наредба v идва преди u . Следователно пресмятаме в **обратна топологическа наредба** (терминалните върхове първи).

```
CountIncreasingPaths(G = (V, E), val[1..n]):
@1  topo[1..n] <- TopologicalSort(G)           // напр. чрез DFS,  $O(V + E)$ 
@2  dp[1..n] <- malloc(n){0}
@3  for v in reverse(topo):                     // обратна топологическа наредба
@4      dp[v] <- 1                             // тривиалния път от един връх
@5      for each edge (v, u) in E:
@6          if val[u] > val[v]:
@7              dp[v] <- dp[v] + dp[u]
@8          endif
@9      endfor
@10 endfor
@11 total <- 0
@12 for v from 1 to n:
@13     total <- total + dp[v]
@14 endfor
@15 return total
```

Сложност:

- Топологическа сортировка: $O(V + E)$.
- Главният цикъл: всеки връх се посещава веднъж, всяко ребро се преглежда веднъж $\Rightarrow O(V + E)$.
- Финалното сумиране: $O(V)$.

Общо: $O(V + E)$ по време, $O(V)$ по памет (плюс паметта за съхранение на графа).

Защо рекурентната връзка е коректна?

Всеки строго нарастващ път, започващ от v , или е тривиален (само v), или има точно първо ребро (v, u) . В последния случай условието „строго нарастващ“ изисква $\text{val}[u] > \text{val}[v]$, а останалата част от пътя $(u \rightarrow \dots)$ е сама по себе си строго нарастващ път, започващ от u .

Защо няма цикли в зависимостите? В DAG няма ориентиран цикъл, така че рекурсията завършва. Формално: топологическата наредба дава линейна подредба, в която v зависи само от върхове, идващи след него — т.е. обратната топологическа наредба е валиден ред за пресмятане.

Коректност (индукция по позицията в обратната топологическа наредба)

Твърдение: $dp[v]$ е броят на строго нарастващите пътища, започващи от v .

Нека $v_1, v_2, \dots, v_n$ е топологическа наредба на G (т.е. за всяко ребро $(v_i, v_j) \in E$ имаме $i < j$). Извършваме индукция по позицията i , обхождайки върховете от $i = n$ надолу към $i = 1$.

База ($i = n$): v_n е последен в топологическата наредба, следователно няма изходящи ребра (всички съседни трябва да са след v_n , но няма такива). Единственият строго нарастващ път от v_n е тривиалният — 1 на брой. Формулата дава $dp[v_n] = 1 + 0 = 1$.

Индукционна стъпка ($i < n$): Предполагаме, че $dp[v_j]$ е коректно за всички $j > i$. Разглеждаме $v = v_i$. Всеки съсед u , такъв че $(v, u) \in E$, задължително идва след v в топологическата наредба, т.е. $u = v_j$ за някое $j > i$. По индукционна хипотеза $dp[u]$ е коректно.

Нека $S(v)$ е множеството от строго нарастващи пътища, започващи от v . Разделяме:

- $S_0(v) = \{(v)\}$ — тривиалния път, $|S_0(v)| = 1$.
- $S_1(v)$ — пътищата с дължина ≥ 1 . Всеки такъв път е от вида $v \rightarrow u \rightarrow \dots$, където $(v, u) \in E$ и $\text{val}[u] > \text{val}[v]$.

За фиксирано допустимо ребро (v, u) (с $\text{val}[u] > \text{val}[v]$), пътищата във вида $v \rightarrow u \rightarrow \dots$ са в биекция с пътищата от $S(u)$ чрез премахване на първото ребро. Така:

$$|S_1(v)| = \sum_{\substack{(v,u) \in E \\ \text{val}[u] > \text{val}[v]}} |S(u)| = \sum_{\substack{(v,u) \in E \\ \text{val}[u] > \text{val}[v]}} dp[u]$$

(в последното равенство използваме индукционната хипотеза).

Следователно:

$$|S(v)| = |S_0(v)| + |S_1(v)| = 1 + \sum_{\substack{(v,u) \in E \\ \text{val}[u] > \text{val}[v]}} dp[u] = dp[v]$$

Накрая, тъй като всеки строго нарастващ път започва от точно един връх, имаме:

$$\text{Total} = \sum_{v \in V} |S(v)| = \sum_{v \in V} dp[v]$$

което е коректно.

Пример

Нека $V = \{1, 2, 3, 4\}$ със стойности $\text{val} = [1, 3, 2, 4]$ и ребра:

$$E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$$

Една валидна топологическа наредба е 1, 2, 3, 4, тоест обратната е 4, 3, 2, 1.

Пресмятане в обратен ред:

- $dp[4] = 1$ (терминален връх).
- $dp[3] = 1 + dp[4] = 2$ (ребро $(3, 4)$, $\text{val}[4] = 4 > 2 = \text{val}[3]$).
- $dp[2] = 1 + dp[4] = 2$ (ребро $(2, 4)$, $\text{val}[4] = 4 > 3 = \text{val}[2]$).
- $dp[1] = 1 + dp[2] + dp[3] = 5$ (ребра $(1, 2)$: $3 > 1$ и $(1, 3)$: $2 > 1$).

v	1	2	3	4
$\text{val}[v]$	1	3	2	4
$\text{dp}[v]$	5	2	2	1

Отговор: $\text{Total} = 5 + 2 + 2 + 1 = 10$.

Проверка чрез изброяване на всички пътища:

- Дължина 0: (1), (2), (3), (4) — 4 пътя.
- Дължина 1: $1 \rightarrow 2$ ($1 < 3$), $1 \rightarrow 3$ ($1 < 2$), $2 \rightarrow 4$ ($3 < 4$), $3 \rightarrow 4$ ($2 < 4$) — 4 пътя.
- Дължина 2: $1 \rightarrow 2 \rightarrow 4$ ($1 < 3 < 4$), $1 \rightarrow 3 \rightarrow 4$ ($1 < 2 < 4$) — 2 пътя.

Общо: $4 + 4 + 2 = 10$.

Обобщение

Задача	Подзадача	Сложност
House Robber	$\text{dp}[i]$ — макс. сума при използване на първите i къщи	$O(n)$
Нарастващи пътища в DAG	$\text{dp}[v]$ — брой строго нарастващи пътища от v	$O(V + E)$

И в двете задачи ключът е стандартен: *кое е последното решение?* При House Robber това е „обираме ли къща i ”; при пътищата в DAG — „кое е *първото* ребро на пътя”. И в двата случая подзадачите са строго по-малки (намалява i , респ. изкачваме се по топологическата наредба), което гарантира коректност и линейна работа.

Задача 3: Време за разпространение на сигнал (Network Delay Time)

Условие: Дадена е мрежа от n възела, номерирани от 1 до n , и m насочени ребра (u_i, v_i, w_i) , означаващи, че сигнал от u_i до v_i се предава за w_i единици време. Изпращате сигнал от възел k . Определете минималното време, за което сигналът ще достигне до *всички* възли. Ако някой възел е недостижим, върнете -1 .

Вход: Списък от m тройки (u_i, v_i, w_i) с $w_i \geq 0$, цяло число n и цяло число k ($1 \leq k \leq n$).

Изход: Минималното време за достигане на сигнала до всички върхове, или -1 , ако някой възел е недостижим.

Мотивация

Сигналът, изпратен от възел k , се разпространява *паралелно* по всички възможни пътища: всяко ребро (u, v, w) е „проводник”, по който сигналът преминава за w единици време. Възел v получава сигнала в момента, в който *най-ранният* сигнал стигне до него — т.е. в момент равен на дължината на **най-късия** (по сума на теглата) път от k до v .

Нека $d(k, v)$ е дължината на най-късия път от k до v (или $+\infty$, ако v е недостижим). Тъй като всички възли получават сигнала паралелно, а ние търсим момента, в който *последният* го получава, отговорът е:

$$T = \max_{v \in V} d(k, v)$$

Ако някое $d(k, v) = +\infty$, то v никога не получава сигнала — връщаме -1 .

Ключово наблюдение: Задачата се свежда до пресмятане на най-късите пътища от един източник до всички останали върхове в насочен граф с неотрицателни тегла — класическа постановка за алгоритъма на Dijkstra.

Идея

Използваме **алгоритъма на Dijkstra**, който за даден стартов връх k връща масив $\text{dist}[1..n]$, такъв че $\text{dist}[v] = d(k, v)$ за всеки връх v (или $\text{dist}[v] = +\infty$, ако v е недостижим от k). При имплементация с бинарна приоритетна опашка алгоритъмът работи за $O((V + E) \log V)$.

Защо Dijkstra е приложим в нашия граф? Алгоритъмът работи коректно *само* върху графи с **неотрицателни** тегла. Това е така, защото основният му инвариант гласи: „когато за пръв път извлечем връх

u от приоритетната опашка, текущото $\text{dist}[u]$ вече е окончателно”. Аргументът е, че всеки алтернативен път до u минава през още нефинализиран връх v с $\text{dist}[v] \geq \text{dist}[u]$ (иначе v щеше да е по-напред в опашката), а оттам нататък разстоянието може само да *нараства*, понеже всички тегла са ≥ 0 . При наличие на отрицателно ребро това разсъждение се проваля, тъй като сума с отрицателни събираеми може да *намали* обща дължина.

В нашата задача е дадено $w_i \geq 0$, така че Dijkstra е напълно приложим. (Ако имаше отрицателни тегла, бихме били принудени да използваме Bellman-Ford с $O(V \cdot E)$, или Johnson за всички двойки.)

След пресмятане на dist , отговорът се извлича с едно линейно обхождане: ако всички разстояния са крайни, връщаме $\max_v \text{dist}[v]$; иначе връщаме -1 .

Стъпки

1. **Избираме да представим графа като списък на съседство.** За всеки връх u съхраняваме списък от двойки (v, w) , такива че (u, v, w) е ребро от входа. Това представяне е удобно за Dijkstra, понеже алгоритъмът се интересува главно от *изходящите ребра* на даден връх, а списъкът на съседство ни дава директен достъп до тях за време $O(\deg(u))$. Алтернативата — матрица на съседство — би коствала $O(V^2)$ памет и $O(V)$ време за обхождане на съседите на всеки връх, което би вдигнало общата сложност до $O(V^2)$ дори при разреден граф.

Важно: графът е *насочен*, така че ребро (u, v, w) *не* води автоматично до (v, u, w) — добавяме само в посоката, в която е дадено във входа. Изграждането отнема $O(V + E)$ време и памет.

2. **Пускаме алгоритъма на Dijkstra** от стартовия връх k . Алгоритъмът връща масив $\text{dist}[1..n]$, в който $\text{dist}[v]$ е дължината на най-късия път от k до v , или $+\infty$, ако v е недостижим. Това е точно информацията, която ни трябва, понеже времето на пристигане на сигнала във връх v съвпада с дължината на най-късия път от k до v .

Алгоритъмът работи с приоритетна опашка по минимум, в която съхранява текущите кандидати за разстояние, и в основния си цикъл итеративно извлича непосетен връх с най-малко текущо разстояние, маркира го като финализиран и релаксира изходящите му ребра. При имплементация с бинарна купа сложността е $O((V + E) \log V)$. Условието $w_i \geq 0$ от задачата ни гарантира, че алгоритъмът ще даде верни най-къси разстояния.

3. **Обхождаме резултата, за да намерим максимума.** След като Dijkstra приключи, в dist имаме времето на пристигане на сигнала във всеки връх. Правим едно линейно обхождане на масива:

- **Ако срещнем** $\text{dist}[v] = +\infty$ за някой v — върхът е недостижим от k , сигналът никога не стига до него. Връщаме -1 , както изисква условието. Можем да прекъснем веднага, без да обхождаме останалите елементи.
- **Иначе** (всички разстояния са крайни), връщаме $\max_v \text{dist}[v]$ — времето, в което *последният* възел получава сигнала. Това е моментът, в който мрежата е напълно покрита.

Тази стъпка отнема $O(V)$ време.

Защо взимаме максимум, а не сума или минимум?

Сигналите се разпространяват *паралелно* — ние *не* изчакаме един път да завърши, преди да започне следващият. Възел v получава сигнала в момента, в който най-бързата „вълна“ стигне до него, а това е $\text{dist}[v] = d(k, v)$. За да отговорим „кога *всички* възли вече са получили сигнала“, трябва да изчакаме *най-бавния* получател — т.е. този с максимално $\text{dist}[v]$. Сумата няма смисъл (тя би била обща „работа“, не време), а минимумът отговаря на времето на *първото* получаване (което е тривиалното 0 при $v = k$).

Коректност

Коректността следва от два факта:

1. **Dijkstra връща верни най-къси разстояния.** Това е стандартен резултат, валиден *при условие*, че *всички тегла са ≥ 0* . Накратко: когато връх u се извлече за пръв път от опашката, всеки алтернативен път до u минава през още нефинализиран връх v с $\text{dist}[v] \geq \text{dist}[u]$, и оттам нататък разстоянието може само да расте (всички тегла ≥ 0). Следователно $\text{dist}[u]$ е окончателно. В нашата задача $w_i \geq 0$, така че алгоритъмът е приложим.

2. Постобработката дава верния отговор. Времето на пристигане на сигнала във връх v е точно $d(k, v) = \text{dist}[v]$. Ако някое $\text{dist}[v] = +\infty$, върхът е недостижим — връщаме -1 съгласно условието. Иначе всички възли са обслужени и последният получава сигнала в момент $\max_v \text{dist}[v]$ — което е търсеният отговор.

Времева сложност

- **Изграждане на списъка със съседни:** $O(V + E)$.
- **Dijkstra:** всеки връх се извлича *валидно* (с актуално разстояние) най-много веднъж и при това обхожда всичките си изходящи ребра. Всяка релаксация може да добави нов запис в приоритетната опашка, така че тя съдържа общо $O(E)$ записа в живота на алгоритъма. Всяка операция push / pop струва $O(\log E) = O(\log V)$ (тъй като $E \leq V^2$, имаме $\log E \leq 2 \log V$). Сумарно: $O((V + E) \log V)$.
- **Линейно обхождане за максимума:** $O(V)$.

Обща сложност: $O((V + E) \log V)$ по време, $O(V + E)$ по памет.

Пример

Нека $n = 4$, $k = 2$ и ребрата са:

$$E = \{(2, 1, 1), (2, 3, 1), (3, 4, 1), (1, 4, 4)\}$$

(Сигнал от 2 до 1 отнема 1; от 2 до 3 отнема 1; от 3 до 4 отнема 1; от 1 до 4 отнема 4.)

Изпълнение на Dijkstra от $k = 2$:

1. **Начало:** $\text{dist} = [\infty, 0, \infty, \infty]$; $Q = \{(0, 2)\}$.
2. Извличаме $(0, 2)$. Релаксираме $(2, 1, 1)$: $\text{dist}[1] \leftarrow 1$. Релаксираме $(2, 3, 1)$: $\text{dist}[3] \leftarrow 1$.
 $\text{dist} = [1, 0, 1, \infty]$; $Q = \{(1, 1), (1, 3)\}$.
3. Извличаме $(1, 1)$. Релаксираме $(1, 4, 4)$: $\text{dist}[4] \leftarrow 5$.
 $\text{dist} = [1, 0, 1, 5]$; $Q = \{(1, 3), (5, 4)\}$.
4. Извличаме $(1, 3)$. Релаксираме $(3, 4, 1)$: $\text{dist}[3] + 1 = 2 < 5 = \text{dist}[4]$, обновяваме $\text{dist}[4] \leftarrow 2$.
 $\text{dist} = [1, 0, 1, 2]$; $Q = \{(2, 4), (5, 4)\}$.
5. Извличаме $(2, 4)$. Връх 4 няма изходящи ребра — няма какво да се релаксира.
6. Извличаме $(5, 4)$. Но $5 > \text{dist}[4] = 2$ — остарял запис, прескачаме.
7. Q е празна, цикълът завършва.

v	1	2	3	4
$\text{dist}[v]$	1	0	1	2

Всички възли са достижими; отговорът е $\max\{1, 0, 1, 2\} = 2$.

Отговор: $T = 2$ — във време 2 всички възли са получили сигнала. (Възел 4 го получава по пътя $2 \rightarrow 3 \rightarrow 4$ с обща дължина 2, *не* по $2 \rightarrow 1 \rightarrow 4$ с дължина 5 — тук виждаме защо обикновен BFS не работи: реброто $1 \rightarrow 4$ изглежда „късо“ откъм брой стъпки, но е „дълго“ откъм време.)